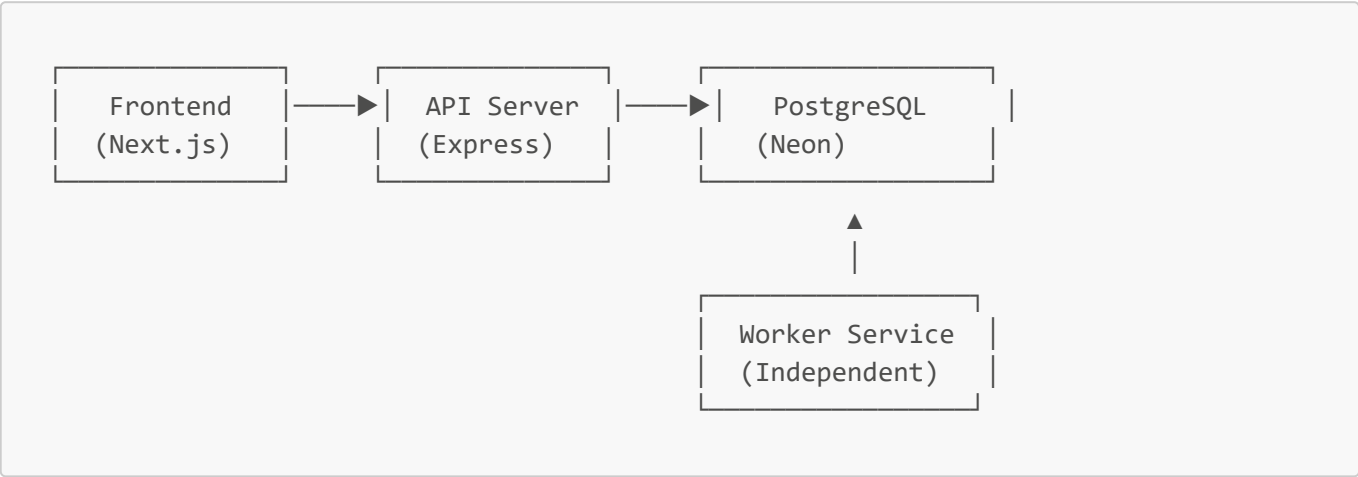


Codity - Distributed Job Scheduler

A production-grade distributed job scheduling platform built with Node.js, TypeScript, PostgreSQL, and Next.js. Inspired by systems like Sidekiq, BullMQ, Celery, and AWS SQS.

Architecture Overview



Components

Component	Technology	Purpose
API Server	Express + TypeScript + Prisma	REST API for job management
Worker Service	Node.js + TypeScript + Prisma	Job execution engine
Frontend	Next.js + TailwindCSS	Monitoring dashboard
Database	PostgreSQL (Neon)	Persistent state store

Quick Start

Prerequisites

- Node.js 20+
- PostgreSQL 15+ (or Neon account)
- npm or yarn

1. Clone and Install

```
# Install backend dependencies
cd backend
npm install
cp .env.example .env
# Edit .env with your database URL

# Install worker dependencies
```

```
cd ../worker
npm install
cp .env.example .env

# Install frontend dependencies
cd ../frontend
npm install
```

2. Database Setup

```
cd backend
npx prisma db push
npx prisma generate
npm run prisma:seed
```

This loads **demo data** for every dashboard page (jobs in all statuses, workers, DLQ entries, audit logs, analytics history, and more).

Demo login: `demo@codity.dev` / `password123`

Demo project ID: `00000000-0000-0000-0000-000000000001`

3. Start Services

```
# Terminal 1: API Server
cd backend && npm run dev

# Terminal 2: Worker Service
cd worker && npm run dev

# Terminal 3: Frontend
cd frontend && npm run dev
```

4. Access

- **API:** `http://localhost:4000`
- **API Docs:** `http://localhost:4000/api/docs`
- **Dashboard:** `http://localhost:3000`

Project Structure

```
codity/
├── backend/                # Express API Server
│   ├── prisma/            # Database schema & migrations
│   ├── src/
│   │   ├── config/        # Configuration management
│   │   └── controllers/    # Request handlers
```

```
├── middleware/ # Auth, validation, error handling
├── routes/     # Express route definitions
├── services/   # Business logic layer
├── types/      # TypeScript type definitions
├── utils/      # Shared utilities
├── tests/      # Unit & integration tests
├── worker/     # Independent Worker Service
├── prisma/     # Shared schema reference
├── src/
│   ├── config/ # Worker configuration
│   ├── services/ # Core worker logic
│   ├── strategies/ # Retry strategy implementations
│   └── utils/    # Worker utilities
├── frontend/   # Next.js Dashboard
│   └── src/
│       ├── app/ # App Router pages
│       ├── components/ # Reusable UI components
│       └── lib/   # API client & utilities
└── docs/       # Additional documentation
```

Key Features

- **Atomic Job Claiming:** Uses PostgreSQL `FOR UPDATE SKIP LOCKED` for conflict-free job distribution
- **Multiple Job Types:** Immediate, Delayed, Scheduled, Recurring (cron), Batch
- **Retry Strategies:** Fixed, Linear, and Exponential backoff with jitter
- **Worker Heartbeats:** Automatic liveness detection and stale worker recovery
- **Dead Letter Queue:** Permanent failures preserved for manual review
- **Graceful Shutdown:** In-flight jobs drain before worker stops
- **Idempotent Execution:** Idempotency keys prevent duplicate job creation
- **Multi-tenant:** Project-based isolation with JWT authentication

Running Tests

```
cd backend
npm test
```

Deployment

Backend (Render)

- Set `DATABASE_URL`, `JWT_SECRET` environment variables
- Build: `npm run build`
- Start: `npm start`

Frontend (Vercel)

- Set `NEXT_PUBLIC_API_URL` to your Render backend URL
- Framework: Next.js (auto-detected)

Database (Neon)

- Create a Neon project
- Run migrations: `npx prisma migrate deploy`

License

MIT